

VerifyAI

Decoding the web, one fact at a time.

COMP 491: Team Future Group

Rainer Santos, Rajvir, Tochi, Jay Patel

Meet The Team! :)

Rainer Santos

Full Stack Dev

- UI/UX Design
- Full Stack Integration
- Frontend App Functionality

Tochi

AI Engineer

- API/AI design
- Full Stack Integration
- Credibility score Logic

Rajvir

Project / API Manager

- Backend APIs
- Database
- Architecture Design

Jay Patel

Database Manager

- Performed testing
- Worked with Azure

Individual Contributions

Rainer :

Built the React (Vite) interface, defined core-logic flow and UI design. Connected and deployed webapp on vercel.

Tochi :

I designed the AI prompts, connected the frontend to the backend, and built the credibility score logic that converts the AI's confidence into a percentage shown alongside each verdict.

Raj :

Worked on frontend and backend. Worked on the database as well as with azure services for cloud apis

Jay :

Deployed services to Azure, performed end-to-end testing via Postman, and resolved integration bugs.

VerifyAI : Project Vision

Problem : Misinformation in Media

Scale: Misinformation spreads faster than manual, traditional fact-checking can operate.

Speed: Fakes go viral instantly and human fact-checking can be slow and unreliable.

Lack of Insight : Users often trust Media & even AI chatbots whole-heartedly instead of independently verifying claims.

Mission : Bridging Back Truth in Media

Empower : Leverage AI for real-time claim verification and critical information awareness.

Explain: Provide full transparency with clear source lineage and credibility reasoning

Discover: Find new sources for elevated media literacy and browse through popular claims

Key Takeaway :

Users are unknowingly caught off guard by misleading online content every single day. VerifyAI promotes media literacy and leverages AI-driven fact-checking to instantly illuminate false claims and misinformation.

VerifyAI : Core Functionality



Browse Claims

Discover commonly verified claims and popular stories happening in real-time.



Media Literacy

Provides recommendations for transparent media sources and key information to stay vigilant online



AI Fact-Checking

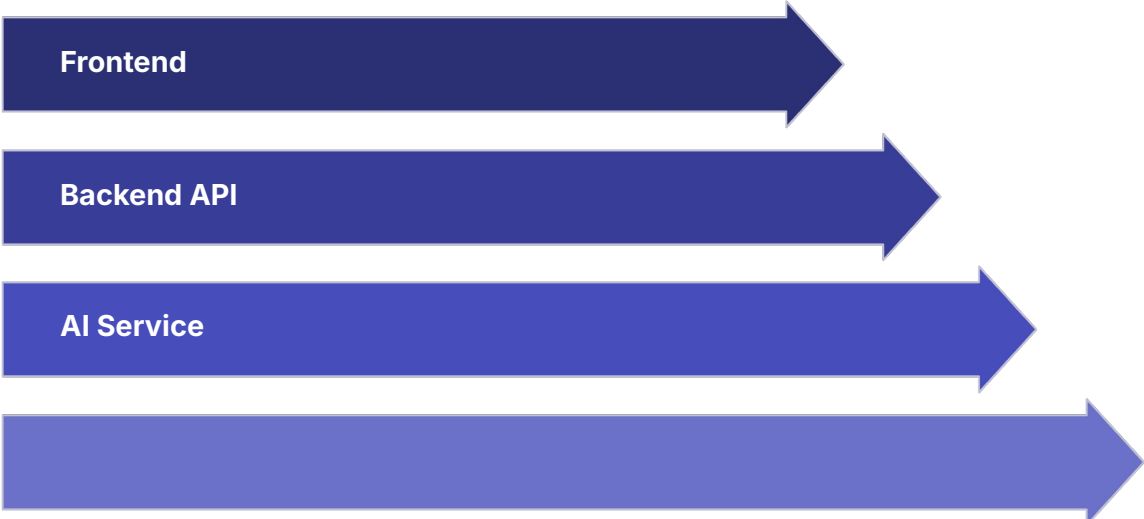
Evaluate statements with AI leveraged fact-checking and the model's detailed reasoning.



Archives

Understand the validity behind AI claims and its reasoning provided with confidence scores.

VerifyAI : Tech Stack



Frontend

Backend API

AI Service

Frontend

Typescript, CSS, React, Python, Azure SWA

Backend

Azure Web Services, Vercel, Node.js

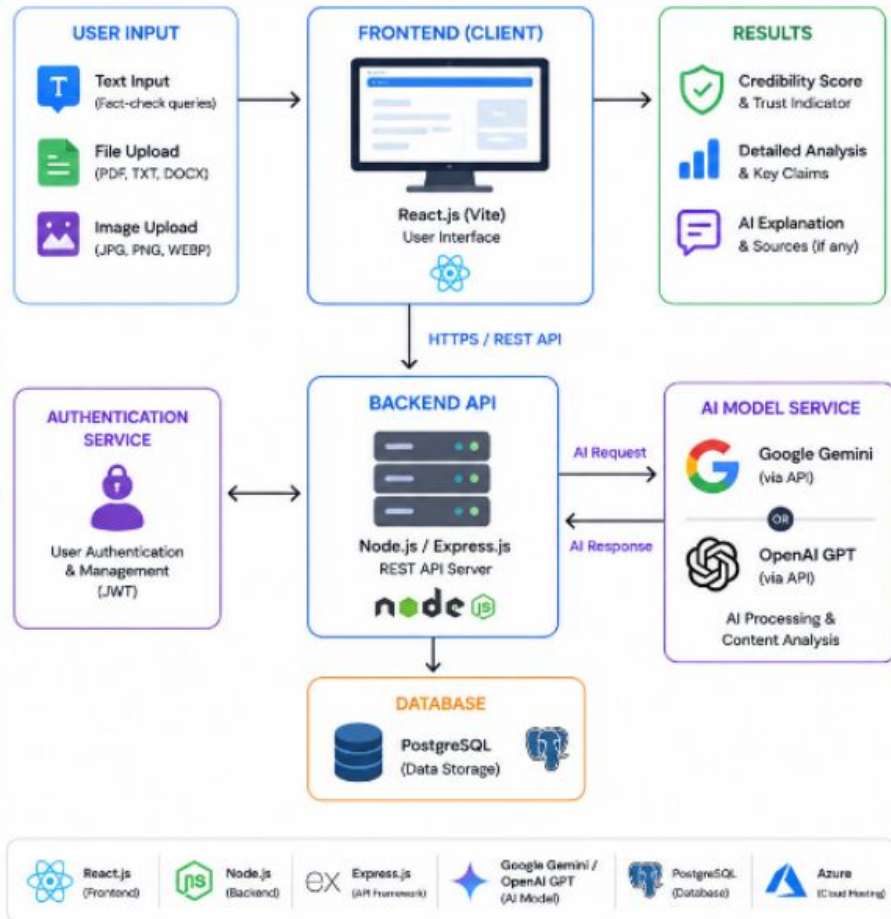
AI Layer

gpt4.0, Gemini 3 Flash, Gemini 3.1 Pro,
FigmaMake

Misc.

Github, Milio, Jira

System Architecture Design



System Components

- Frontend built with React provides the user interface
- Backend API (Node.js / Express) handles requests and processing
- AI service performs analysis and generates responses
- Database stores user data and supports system scalability

Data Flow Representation

- User submits input (text, file, or image) through the UI
- Frontend sends request to backend via API
- Backend processes request and forwards it to AI
- AI returns analysis results to backend
- Backend sends structured response to frontend

Technology Stack Legend

- React (Frontend UI)
- Node.js / Express (Backend API)
- Azure OpenAI/Google Gemini API (AI Processing)
- PostgreSQL (Data Storage)
- Azure (Cloud Deployment)

How the AI works

When a user types a claim, the app sends it to an AI model (GPT-4.0) that reads it, reasons about it, and replies with a verdict.

1

User types a claim

Example: "The Eiffel Tower is in London." The frontend sends this text to our backend server.

2

Backend talks to the AI

Our server sends the claim to GPT-4.0 with instructions: "You are a fact-checker. Return true or false and explain why."

3

AI responds with a verdict

GPT-4.0 replies with a structured answer: verdict (TRUE/FALSE), a confidence score, and a short explanation.

4

User sees the result

The frontend displays the verdict, confidence percentage, and evidence — all in under a few seconds.

THE APIS WE USED

An API is how two pieces of software talk to each other. This app uses three main APIs to function.



GPT-4.0 AI API

The brain of the app

What it does:

We send it a claim and it sends back a verdict. This is the core of the whole project — without this API, there is no fact-checking.

How we call it:

We include the user's claim and instructions telling the AI to respond in JSON format with a verdict and confidence score.



Azure API Management

The security checkpoint

What it does:

Sits in front of our backend and checks that every request is legitimate before it reaches the AI. Blocks bad actors, enforces limits.

How we use it:

Every call from the frontend passes through this layer. It validates the user token and injects the AI API key from Key Vault.



Frontend ↔ Backend REST API

The internal connector

What it does:

This is our own API — the React frontend uses it to send claims to the backend and get verdicts back. We built this ourselves.

How we call it:

React calls this endpoint using `fetch()`. The backend handles it, calls GPT-4.0, then returns the verdict back to the UI.

API Design

Endpoint	Method	Purpose	Description
<code>/api/registerUser</code>	POST	Register user	Creates a new user account with email and password
<code>/api/loginUser</code>	POST	Authenticate user	Verifies user credentials and returns a unique user ID
<code>/api/submitText</code>	POST	Submit text claim	Stores user input text and generates a submission ID
<code>/api/submitURL</code>	POST	Submit URL	Accepts a URL and prepares it for content analysis
<code>/api/submitImage</code>	POST	Submit image	Uploads an image and performs initial AI-based analysis
<code>/api/runAnalysis</code>	POST	Run AI analysis	Processes submitted claim using AI and returns results
<code>/api/getAnalysis/{id}</code>	GET	Get analysis result	Retrieves detailed analysis results using submission ID
<code>/api/getHistory</code>	GET	Fetch user history	Returns all previous claims and analysis results for a user

JSON Request / Response

API Example



Request

```
POST /api/ai-chat
Content-Type: application/json

{
  "message": "Is the earth flat?"
}
```



Response

```
{
  "status": "success",
  "reply": "The Earth is not flat.
It is an oblate spheroid,
confirmed by satellite imagery,
physics, and centuries of
scientific observation."
}
```

Design Principles



Consistent Structure

Every response follows a uniform JSON schema with a `status` field for easy frontend handling.



Descriptive Outputs

Fact-check endpoints return a `credibility_score`, `reasoning`, and `key_claims` array — not just a binary result.



Error Handling

Invalid or empty inputs return structured error messages, keeping the frontend experience smooth.

Cloud APIs

System Architecture

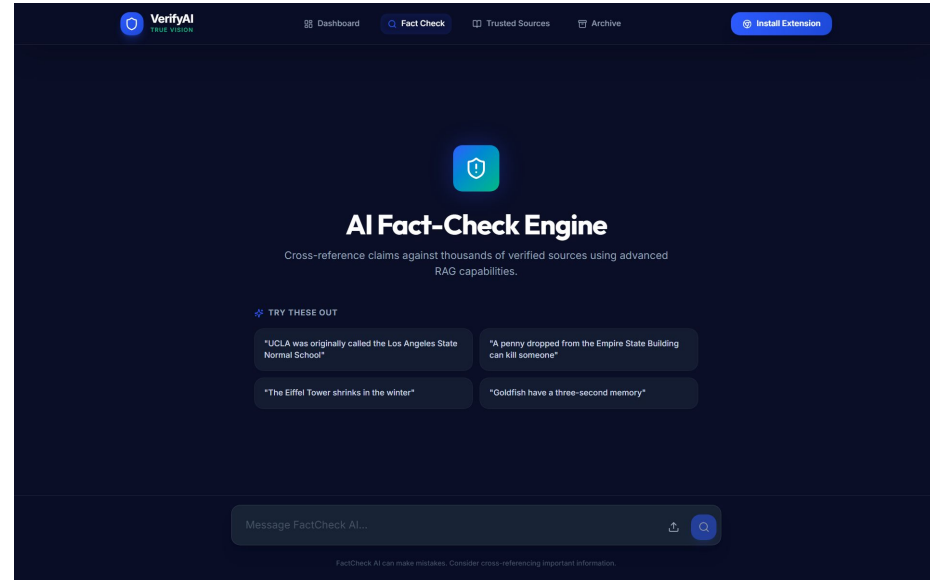
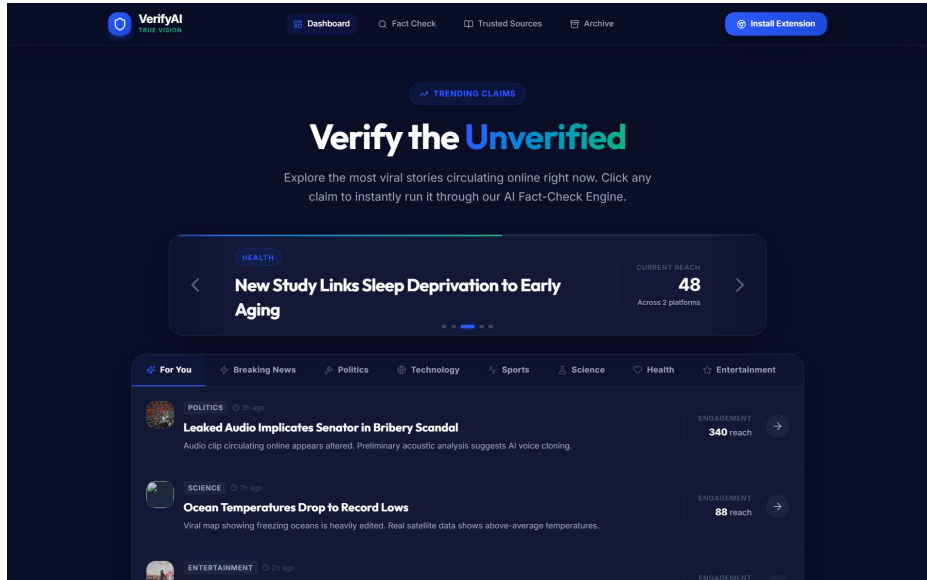
User → Frontend → Flask API → Azure OpenAI → PostgreSQL → Response

Cloud API Example

Endpoint: POST /api/runAnalysis

Purpose: Analyze user-submitted claim using Azure OpenAI (Cloud API)

UI/UX Design



Testing & Validation

Testing Methods

01

Postman API

Testing

Each endpoint was tested individually with valid, invalid, and edge-case inputs to verify correct behavior.

02

Frontend Validation

Manual testing performed locally and on the deployed cloud environment for both desktop and upload flows.

03

File & Image Upload

Testing

PDF, TXT, DOCX, and image formats validated for proper parsing and accurate AI analysis results.

Improvements Made

API Response Optimization

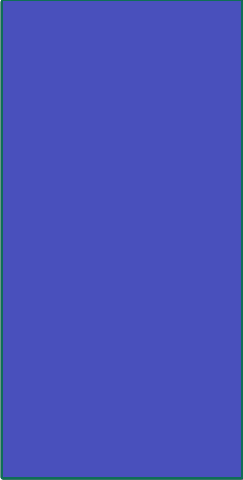
Reduced latency and standardized response structure across all endpoints.

Frontend-Backend Communication

Fixed CORS issues and improved error propagation from API to UI.

AI Reliability

Enhanced prompt design to produce more consistent, structured Gemini API outputs.



VerifyAI : Demo Walkthrough

Conclusion

Our Next Focus

Tokenization Usage

Improve efficiency of AI usage with claim reasoning and improve set rate limits

Image Submission

Integrate AI image detection functionality to determine and analyze AI created media

Future Roadmap

Expand Database

Build out PostgreSQL schema for claim history, , saved AI I/O analytics.

1

2

Improve AI Accuracy

Refine Gemini prompts and explore ensemble approaches for more reliable scoring.

Advanced Analytics

Add trend dashboards, claim tracking, and source credibility over time.

3

4

Scale to Production

Harden security, optimize performance, and prepare for broader user adoption.

Questions & Answers

COMP 491: Team Future Group

Rainer Santos, Rajvir, Tochi, Jay Patel